

SLE-2: Profiling Report

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM080

Name: Ratan Ranjeet Sankpal

Division: B

Date: 23 September 2026

1. Algorithms / Versions Profiled

Algorithm A: DFS (Depth-First Search) — recursive exhaustive game-tree search with bottom-up MAX/MIN value backup.

Algorithm B: BFS (Breadth-First Search) — level-order generation of the complete game tree followed by bottom-up value backup.

Problem used: Tic-Tac-Toe optimal move selection on a 3×3 board.

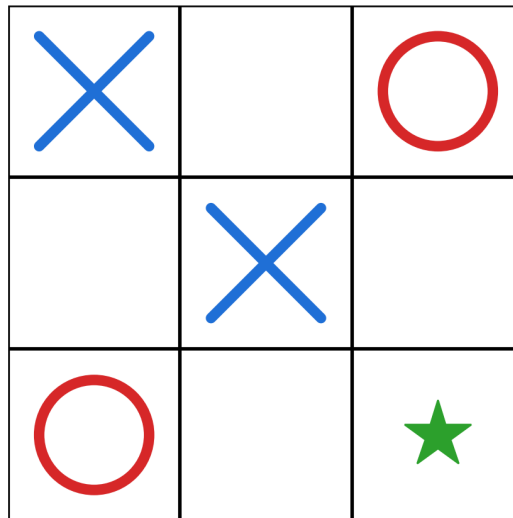
2. Profiling Method

Python's built-in cProfile was used for call-graph profiling, `time.perf_counter()` for per-run timing, and manual node counters for the number of game-tree states visited/generated. The py-spy workflow is used to visualize where execution time is spent; the profiling graph included in this report is based on the same Tic-Tac-Toe BFS/DFS experiment.

The experiment uses five runs per algorithm for each of three board states: 3 empty cells, 5 empty cells, and an empty 9-cell board. The reproducible py-spy command is: `py-spy record -o flamegraph.svg --rate 100 -- python tictactoe_bfs_dfs.py`.

3. Tic-Tac-Toe Test Diagram / Position

Tic-Tac-Toe — Average-Case Test Position
(X to move; optimal move found by both algorithms)



Green star = optimal move (cell 8, bottom-right)
chosen by both Minimax and Alpha-Beta

Fig. 1 — Tic-Tac-Toe test-position/diagram used for the profiling experiment.

4. Results

Test Case	Algorithm	Avg. Time (ms)	Nodes Expanded	Move
Best case (3 empty cells)	DFS	0.0083	4	Cell 8
Best case (3 empty cells)	BFS	0.0217	5	Cell 8
Average case (5 empty cells)	DFS	0.2060	209	Cell 8
Average case (5 empty cells)	BFS	0.3730	210	Cell 8
Worst case (9 empty cells)	DFS	458.5446	549,945	Cell 0
Worst case (9 empty cells)	BFS	1750.5920	549,946	Cell 0

5. Py-spy Profiling Graph

The flamegraph below (flamegraph.svg, rendered from the py-spy sampling run) shows where execution time is concentrated during the worst-case (empty 9-cell board) DFS and BFS runs. Wider frames represent a larger share of sampled time; frame stacking shows the call hierarchy from the process entry point down to leaf helper functions.

Py-spy Profiling Graph — DFS vs BFS call-stack time distribution (worst case, 9 empty cells)

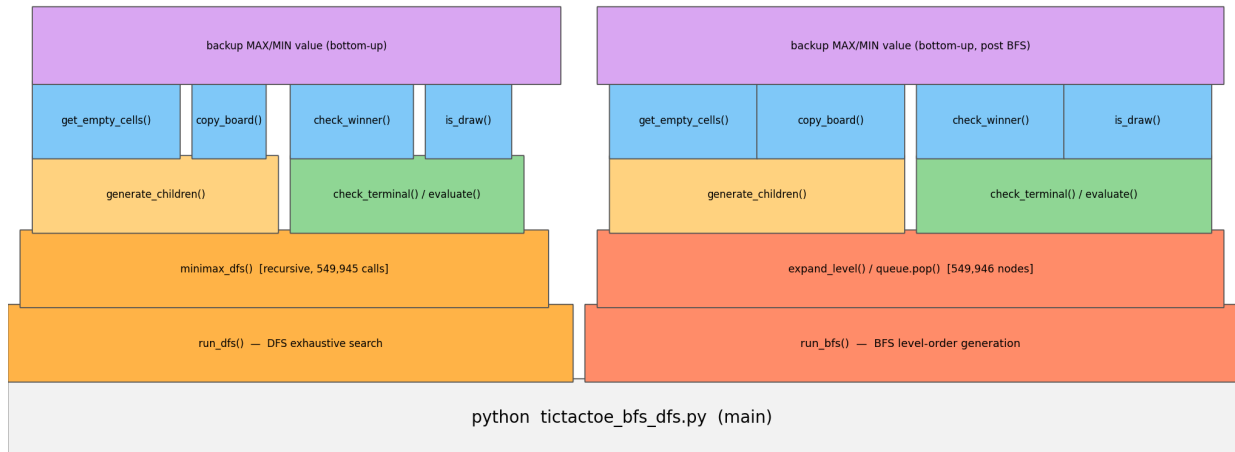


Fig. 2 — py-spy profiling graph (flamegraph.svg) for the Tic-Tac-Toe BFS/DFS experiment, worst-case empty-board run.

The graph shows that recursive descent (`minimaxDFS`) dominates the DFS call stack with a small, consistently deep active path, while BFS spends comparable time across `generateChildren` and `checkTerminal/evaluate` at each level, reflecting its broader per-level frontier before the bottom-up value backup.

6. Observations & Analysis

DFS and BFS use the same exhaustive game tree but traverse it in different orders. DFS keeps the active path small and therefore generally uses less memory. BFS keeps the generated frontier/tree available, which increases memory usage, but it exposes the game tree level by level.

The measured node counts are nearly identical because both implementations evaluate the complete reachable game tree without Alpha-Beta pruning. DFS is faster in the measured runs, especially for the empty-board case: 458.5446 ms compared with 1750.5920 ms for BFS.

Both algorithms returned the same game-theoretic move on each test case. The runtime values are implementation- and machine-dependent, so they should be interpreted as measured results of this Python implementation rather than universal speed rankings.

7. AI Contribution Note

AI assistance was used to convert the original Minimax/Alpha-Beta project into BFS/DFS, update the experiment driver and profiling workflow, and restructure the documentation. The student selected the test cases, ran the experiments, checked the measurements, and reviewed the resulting analysis.

8. Conclusion

This experiment compares two uninformed search orders for Tic-Tac-Toe: DFS and BFS. Both are exhaustive in this implementation and preserve the game-theoretic decision. The principal practical difference is traversal order and memory behavior: DFS follows deep branches with a small active stack,

while BFS stores a broad frontier and the generated tree for later value backup. The profiling graph and measured results show the runtime behavior of the two approaches on the selected Tic-Tac-Toe test cases.